

Morphix: Real-Time GAN Latent Editor in Streamlit

Kshitij Bajpai

Overview

Morphix is a real-time face generator and editor built on top of **StyleGAN2-ADA**. It allows users to generate, edit, and mix faces using interpretable controls powered by precomputed latent directions. The application is fully interactive and deployed using Streamlit, with support for undo/redo, attribute sliders (Smile, Age, Gender), and style mixing.

System Architecture

The architecture comprises three main components:

- **Frontend (Streamlit UI):** Handles user interaction and display of images and controls.
- **Backend (StyleGAN2-ADA):** Generates faces and applies latent space manipulations.
- **Latent Editor Logic:** Applies pretrained latent direction vectors to edit features like beard, glasses, and gender.

The pretrained model (`ffhq.pkl`) is loaded once via a cached function using `@st.cache_resource` to avoid repeated GPU/CPU allocation. All edits operate directly on the $\mathbf{w}^+$ latent space, which has shape `[1, 18, 512]`.

Latent Space Logic

Faces in StyleGAN2 are generated from a latent vector $\mathbf{z} \in \mathbb{R}^{512}$, which is mapped to an intermediate latent code $\mathbf{w}$. For greater control, we use the extended latent space $\mathbf{w}^+$, which allows each of the 18 layers to have its own 512-dimensional vector.

Attribute Edits: Each attribute corresponds to a latent direction $\mathbf{d}_i \in \mathbb{R}^{18 \times 512}$. We perform edits as:

$$\mathbf{w}^+_{\text{edited}} = \mathbf{w}^+_{\text{original}} + \alpha_i \cdot \mathbf{d}_i$$

where α_i is the user-selected scalar from a slider.

Style Mixing: To mix two faces, we combine their latent codes at a selected layer index k :

$$\mathbf{w}^+_{\text{mixed}}[:, k :] = \mathbf{w}^+_2[:, k :], \quad \text{rest from } \mathbf{w}^+_1$$

Undo/Redo Stack: We use two lists in Streamlit's `session_state`: an `undo_stack` and a `redo_stack`, which store copies of previous $\mathbf{w}^+$ states. On each edit, the current state is pushed to `undo_stack`, enabling complete reversible interaction.

UI Overview

The application interface is laid out as follows:

- **Generate Button:** Produces a new random face from Gaussian noise.
- **Image Display:** Shows the current generated or edited face.
- **Undo/Redo Buttons:** Navigate the edit history stack.
- **Attribute Sliders:** Control Smile, Age, Gender attributes.
- **Preset Edits:** One-click buttons to apply binary attribute edits (e.g., Glasses, Cartoon, Artistic).
- **Style Mixer:** Combines the current face with a second sampled face using a crossover slider.
- **Download Button:** Saves the current image locally.

Dockerized Workspace Setup

To ensure reproducibility and a clean environment, I containerized the workspace using a lightweight `python:3.9-slim` base image. Below is the Dockerfile I wrote:

```
FROM python:3.9-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    software-properties-common \
    git \
    wget \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt ./
COPY src/ ./src/
```

```

# Download latent direction vectors into /app/presets
RUN mkdir -p /app/presets && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/eyeglass_boundary.npy -O /app/presets/glasses.npy
  && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/beard_boundary.npy -O /app/presets/beard.npy && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/age_boundary.npy -O /app/presets/age.npy && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/gender_boundary.npy -O /app/presets/gender.npy && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/smile_boundary.npy -O /app/presets/smile.npy && \
  wget https://github.com/genforce/interfacegan/raw/master/
    boundaries/child_boundary.npy -O /app/presets/child.npy && \
  wget https://raw.githubusercontent.com/justinpinkney/stylegan2-
    ada-transformer/main/directions/cartoon.npy -O /app/presets/
    cartoon.npy && \
  wget https://raw.githubusercontent.com/justinpinkney/stylegan2-
    ada-transformer/main/directions/artistic.npy -O /app/presets/
    artistic.npy

RUN pip3 install -r requirements.txt

# Clone StyleGAN2-ADA-PyTorch
RUN git clone https://github.com/NVlabs/stylegan2-ada-pytorch.git /
  app/stylegan2-ada-pytorch
WORKDIR /app/stylegan2-ada-pytorch

# Optional: specific requirements
RUN pip install imageio-ffmpeg==0.4.3 pyspng==0.1.0

# Download pretrained weights
RUN wget https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/
  pretrained/ffhq.pkl -P pretrained/

# Return to /app for Streamlit launch
WORKDIR /app

EXPOSE 8501
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health

ENTRYPOINT ["streamlit", "run", "src/streamlit_app.py", "--server.
  port=8501", "--server.address=0.0.0.0"]

```

With this setup, I can launch the app locally or in a containerized server with a single

command. All required assets — models, directions, and code — are bundled into the image, enabling reproducibility and ease of deployment.

This app can be set run on hugging face spaces, might show an error for CPU usage, in that case try loading the model as float on the device.